Importing the needed libraries

↳ 1 cell hidden

Uploading the data using Kaggle API

↳ 3 cells hidden

Preparing the data

↳ 4 cells hidden

Spliting the data (avoid leak when augmentation)

↳ 7 cells hidden

Data Auggmentation only for training

```
# Create directories to save the augmented data in
for CN in ['NORMAL', 'PNEUMONIA']:
    os.makedirs(os.path.join(aug_path, CN), exist_ok=True)
```

```
# configuration of data augmentation
data_aug_train = ImageDataGenerator(
    rotation_range=20, width_shift_range=0.2, height_shift_range=0.2,
    horizontal_flip=True, zoom_range=0.2, shear_range=0.2, fill_mode='nearest'
)
```

```
img_count = 0
target_count = 3923 #same number as penumonia class to balance data
```

```
# Training data with augmentation
#loop over images in normal folder only
for img_name in os.listdir(os.path.join(train_path, 'NORMAL')):

    if img_count >= target_count: #if num of images exceeds the target stop

        break

    try:

        img = Image.open(os.path.join(os.path.join(train_path, 'NORMAL'), img_name)).convert("RGB")
        img_array = np.expand_dims(np.array(img), axis=0) #expanding the image dimintation so it becomes compatb

        aug_iter = data_aug_train.flow(img_array, batch_size=1)

        for i in range(min(5, target_count - img_count)): #up to five versions, stop if reached target_num

            save_path = os.path.join(os.path.join(aug_path, 'NORMAL'), f"{img_count}_{img_name}")#saving path an

            Image.fromarray(next(aug_iter)[0].astype('uint8')).save(save_path)#from numpy back to

            img_count += 1

            if img_count >= target_count:

                break

    except Exception as e:   #error handling to not stop because of bad images
        print(f"Skipping {img_name}: {e}")
```

```
pneumonia_src = os.path.join(train_path, 'PNEUMONIA')#source
pneumonia_dst = os.path.join(aug_path, 'PNEUMONIA')#distination

for img_name in os.listdir(os.path.join(train_path, 'PNEUMONIA')): #loop through each image from the class pneum
```

```
    #copy image from source to didstination
    shutil.copy2(os.path.join(pneumonia_src, img_name), os.path.join(os.path.join(aug_path, 'PNEUMONIA'), img_na
```

making sure that augmentation went good

```
#making sure that augmentation was done
train_normal_num_aug= len(os.listdir(os.path.join(aug_path,"NORMAL")))
train_pneumonia_aug= len(os.listdir(os.path.join(aug_path, "PNEUMONIA")))
print(f"Total Training images: {train_normal_num_aug + train_pneumonia_aug}")
print(f"Normal: {train_normal_num_aug}, Pneumonia: {train_pneumonia_aug}")
```

```
Total Training images: 7846
Normal: 3923, Pneumonia: 3923
```

```
#downloading the data for backup
shutil.make_archive('/content/augmented_data', 'zip', '/content/augmented_data')
files.download('/content/augmented_data.zip')
```

Data generation and loading

↳ 3 cells hidden

Models initialization as methods to train on different hyperparameters easier

↳ 8 cells hidden

Preparing the parameters and functions to train the models

↳ 8 cells hidden

Models training and best models validation

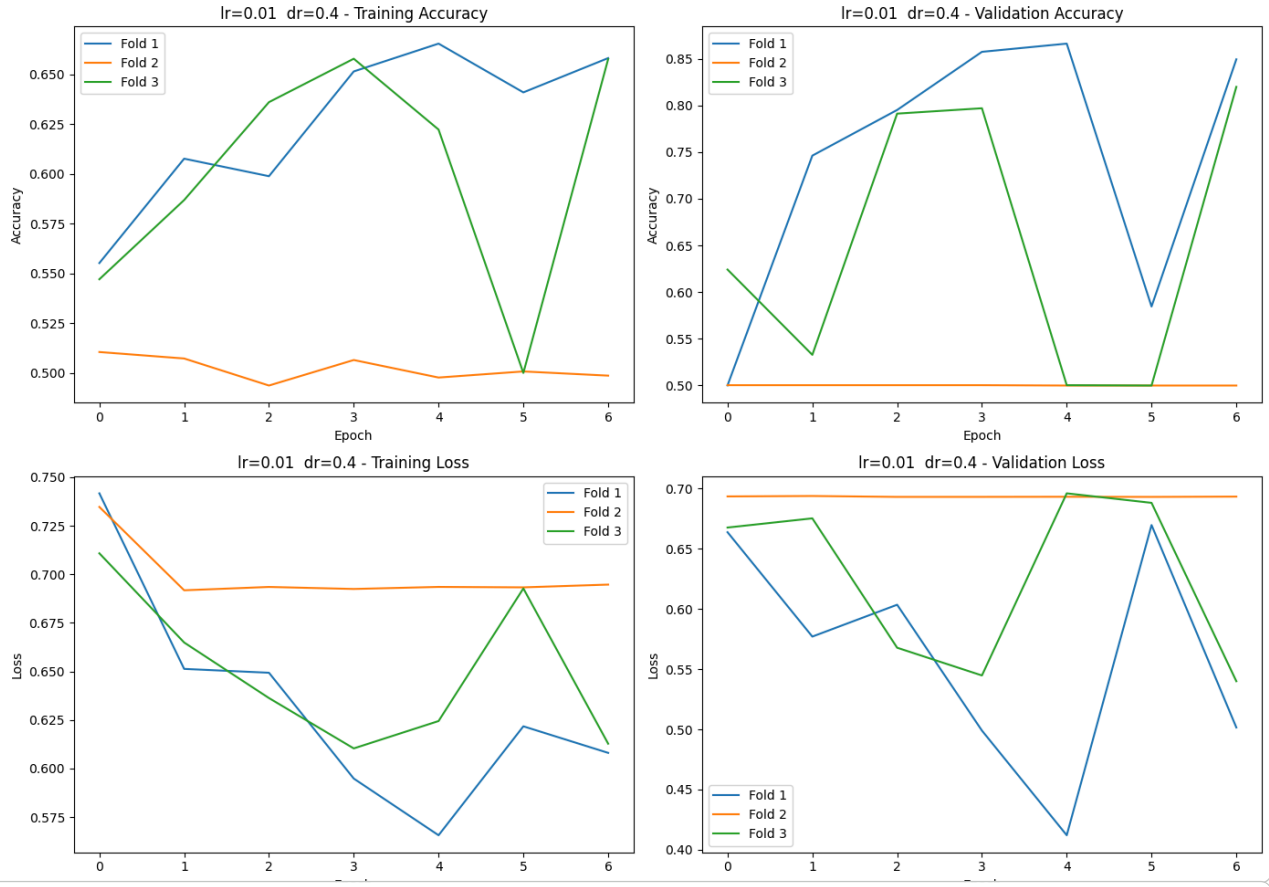
ResNet50

```
#creating a list to save all models result in
Resnet50_model_results = []
```

```
#loop over each combinations
for parameters in hyperparameters:
    #logging start time to calculate time complexity
    ST = time.time()

    print(f"\n Hyperparameters: {parameters}")

    f = 1#track fold num
    training_trackers = [] #to save train accuracy and calculate mean of the folds
    validation_trackers = [] #to save validation accuracy and calculate mean of the folds
    fold_trackers = [] #to save each fold performance to plot


    #loop overk-fold splits
    for train_idx, val_idx in cross_validation.split(X_images, y_label):

        print(f"\n Fold: {f}")

        #split the data for the current fold
        X_train, X_val = X_images[train_idx], X_images[val_idx]
        y_train, y_val = y_label[train_idx], y_label[val_idx]

        #combiling the model using the method created and passing the current hyperparameter combo
        resnet = create_resnet50(learning_rate=parameters["learning_rate"],dropout_rate=parameters["dropout_rate

        #save best model to load it in testing
        model_name = f"Resnet50_fold{f}_lr-{parameters['learning_rate']}_dr-{parameters['dropout_rate']}.h5"
        checkpoint = ModelCheckpoint(model_name, monitor='val_accuracy', save_best_only=True, verbose=0)
```

```
    #patience=3 , meaning if accuracy didn't improve for 3 epochs stop
    #restore_best_weights=True, meaning when it stops it would take the best epochs weights not the last one
    callback = [EarlyStopping(patience=3, restore_best_weights=True), checkpoint]

    #fit the model
    h = resnet.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=7,
        batch_size=32,
        callbacks=callback,
        verbose=0
    )

    #evaluate
    scores = resnet.evaluate(X_val, y_val, verbose=0)
    #add the scores to the trackers to calculate mean
    training_trackers.append(h.history["accuracy"][-1])
    validation_trackers.append(scores[1])
    #to plot
    fold_trackers.append(h.history)

    f += 1
#calculating the time taken to train this model
ET = time.time()
Overall = (ET - ST) / 60  # to minutes
print(f" Time taken to train this model: {Overall:.2f} minutes")

#plot learning curve for all folds
all_folds_learning_curves(fold_trackers, title=f"lr={parameters['learning_rate']}  dr={parameters['dropout_r

#adding scores to the list to better compare
Resnet50_model_results.append({
    "hyperparameters": parameters,
    "training_accuracy": np.mean(training_trackers),
    "validation_accuracy": np.mean(validation_trackers),
    "Training time": Overall,
})
```

```
 Hyperparameters: {'learning_rate': 0.01, 'dropout_rate': 0.4}

 Fold: 1
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th

 Fold: 2
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th

 Fold: 3
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
 Time taken to train this model: 3.23 minutes
```



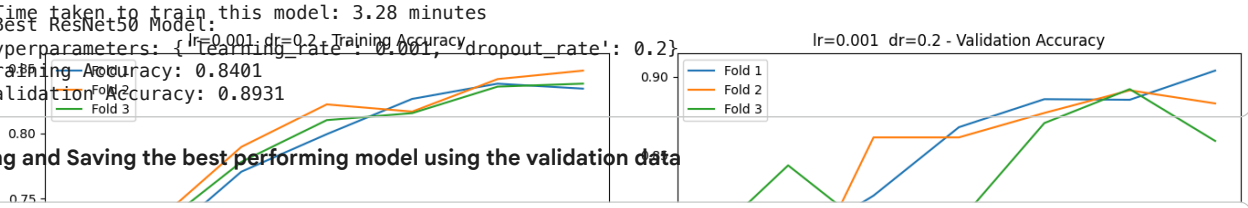

```
Resnet50_model_results= pd.DataFrame(Resnet50_model_results)
```

```
Resnet50_model_results
```

	hyperparameters	training_accuracy	validation_accuracy	Training time
0	{'learning_rate': 0.01, 'dropout_rate': 0.4}	0.604834	0.728634	3.228461
1	{'learning_rate': 0.001, 'dropout_rate': 0.2}	0.840109	0.893065	3.283049
2	{'learning_rate': 0.005, 'dropout_rate': 0.3}	0.798240	0.874330	3.265140

```
#find best model
best_model_Resnet50 = Resnet50_model_results.loc[Resnet50_model_results["validation_accuracy"].idxmax()]
print("\n Best ResNet50 Model:")
print("Hyperparameters:", best_model_Resnet50["hyperparameters"])
print(f"Training Accuracy: {best_model_Resnet50['training_accuracy']:.4f}")
print(f"Validation Accuracy: {best_model_Resnet50['validation_accuracy']:.4f}")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
 Time taken to train this model: 3.28 minutes
 Best ResNet50 Model:
Hyperparameters: {'learning_rate': 0.001, 'dropout_rate': 0.2}
Training Accuracy: 0.8401
Validation Accuracy: 0.8931
```



Training and Saving the best performing model using the validation data

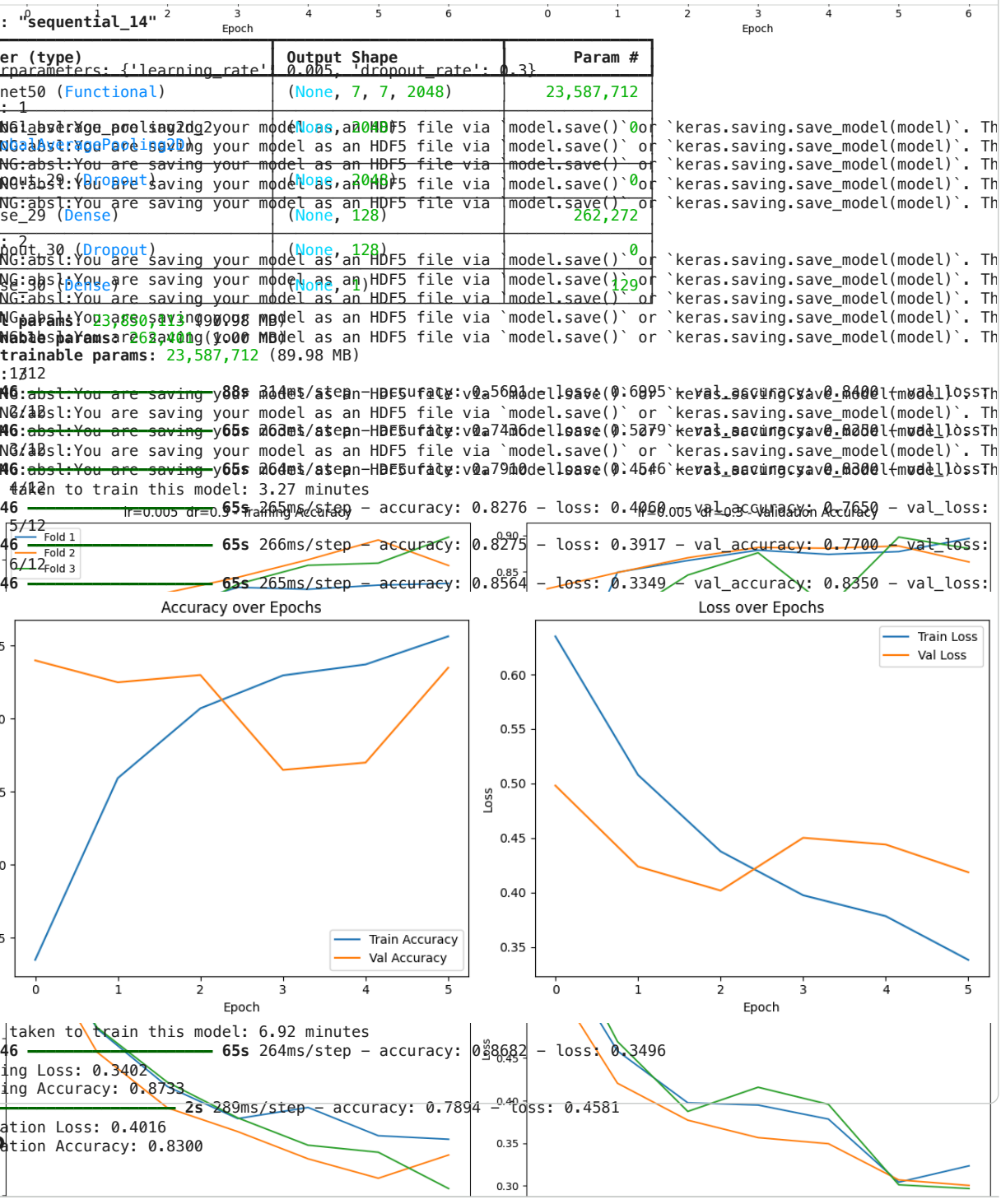
```
best_model_res = create_resnet50(learning_rate=0.001, dropout_rate=0.2)
best_model_res.summary()
ST = time.time()
history = best_model_res.fit(
    train_generator,
    validation_data=val_generator,
    epochs=12,
    callbacks=[EarlyStopping(patience=3, restore_best_weights=True)],
    verbose=1
)
Best_learning_curve(history)

ET = time.time()
Overall = (ET - ST) / 60  # to minutes
print(f" Time taken to train this model: {Overall:.2f} minutes")

train_loss, train_accuracy = best_model_res.evaluate(train_generator, verbose=1)
print(f"Training Loss: {train_loss:.4f}")
print(f"Training Accuracy: {train_accuracy:.4f}")

val_loss, val_accuracy = best_model_res.evaluate(val_generator, verbose=1)
print(f"Validation Loss: {val_loss:.4f}")
print(f"Validation Accuracy: {val_accuracy:.4f}")

best_model_res.save("ResNet50_best.keras")
```

```
Model: "sequential_14"
```

Layer (type)	Output Shape	Param #
resnet50 (Functional)	(None, 7, 7, 2048)	23,587,712
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None, 2048)	0
dropout_29 (Dropout)	(None, 2048)	0
dense_29 (Dense)	(None, 128)	262,272
dropout_30 (Dropout)	(None, 128)	0
dense_30 (Dense)	(None, 1)	129

```
 Total params: 23,850,113 (90.98 MB)
 Trainable params: 262,401 (1.00 MB)
 Non-trainable params: 23,587,712 (89.98 MB)
```

```
Hyperparameters: {'learning_rate': 0.005, 'dropout_rate': 0.3}
Fold: 1
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
Fold: 2
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
Epoch 1/12
246/246 ━━━━━━━━━━━━━━━━━━━━ 88s 314ms/step - accuracy: 0.5601 - loss: 0.6895 - val_accuracy: 0.8400 - val_loss:
Epoch 2/12
246/246 ━━━━━━━━━━━━━━━━━━━━ 65s 263ms/step - accuracy: 0.7436 - loss: 0.5279 - val_accuracy: 0.8250 - val_loss:
Epoch 3/12
246/246 ━━━━━━━━━━━━━━━━━━━━ 65s 264ms/step - accuracy: 0.7910 - loss: 0.4546 - val_accuracy: 0.8300 - val_loss:
Epoch 4/12
246/246 ━━━━━━━━━━━━━━━━━━━━ 65s 265ms/step - accuracy: 0.8276 - loss: 0.4060 - val_accuracy: 0.7650 - val_loss:
Epoch 5/12
246/246 ━━━━━━━━━━━━━━━━━━━━ 65s 266ms/step - accuracy: 0.8275 - loss: 0.3917 - val_accuracy: 0.7700 - val_loss:
Epoch 6/12
246/246 ━━━━━━━━━━━━━━━━━━━━ 65s 265ms/step - accuracy: 0.8564 - loss: 0.3349 - val_accuracy: 0.8350 - val_loss:
```




```
 Time taken to train this model: 6.92 minutes
246/246 ━━━━━━━━━━━━━━━━━━━━ 65s 264ms/step - accuracy: 0.8682 - loss: 0.3496
Training Loss: 0.3402
Training Accuracy: 0.8733
7/7 ━━━━━━━━━━━━━━━━━━━━ 2s 289ms/step - accuracy: 0.7894 - loss: 0.4581
Validation Loss: 0.4016
Validation Accuracy: 0.8300
```

VGG16

↳ 7 cells hidden

AlexNet

↳ 7 cells hidden

DenseNet121

↳ 7 cells hidden

Evaluation of best models of each architecture using test data

```
#creating method to avoid duplicating the code
def evaluate_on_test(model, model_name):
    #start from the beginning of the data
    test_generator.reset()
    #predict we use test_generator to load images batch-by-batch and applies the initialized preprocessing
    predictions = model.predict(test_generator)
    #threshold where if predection> 0.5 will return 1 (Pneumonia) not return 0 (Normal).
    y_pred = (predictions > 0.5).astype(int).flatten() #threshold where if predection> 0.5 will return 1 (Pneumo
    #actual label
    print(predictions[:10])
    y_true = test_generator.classes
    #calculate the metrices and return them
    return {
        'accuracy': accuracy_score(y_true, y_pred),
        'precision': precision_score(y_true, y_pred),
        'recall': recall_score(y_true, y_pred),
        'f1_score': f1_score(y_true, y_pred),
        'confusion_matrix': confusion_matrix(y_true, y_pred)
    }
```

```
#loading the best models of each Architicture
best_models = {
    "ResNet50": load_model("ResNet50_best.keras"),
    "VGG16": load_model("VGG16_best.keras"),
    "AlexNet": load_model("AlexNet_best.keras"),
    "DenseNet": load_model("DenseNet121_best.keras")
}
```

```
#evaluating the model and adding metrices to a list
Testing = {}
for model_name, model in best_models.items():
    print(f"\nEvaluating {model_name}...")
    results = evaluate_on_test(model, model_name)
    Testing[model_name] = results
```

```
Evaluating ResNet50...
/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning:
  self._warn_if_super_not_called()
16/16 ━━━━━━━━━━━━━━━━━━━━ 15s 690ms/step
[[0.1823261 ]
 [0.28791603]
 [0.5130435 ]
 [0.47435057]
 [0.26671943]
 [0.4065797 ]
 [0.39469293]
 [0.5850558 ]
 [0.33275247]
 [0.5489864 ]]

Evaluating VGG16...
16/16 ━━━━━━━━━━━━━━━━━━━━ 8s 424ms/step
[[0.08901034]
 [0.42914057]
 [0.53837407]
 [0.27839443]
 [0.86675614]
 [0.4062    ]
 [0.00683139]
 [0.962977  ]
```

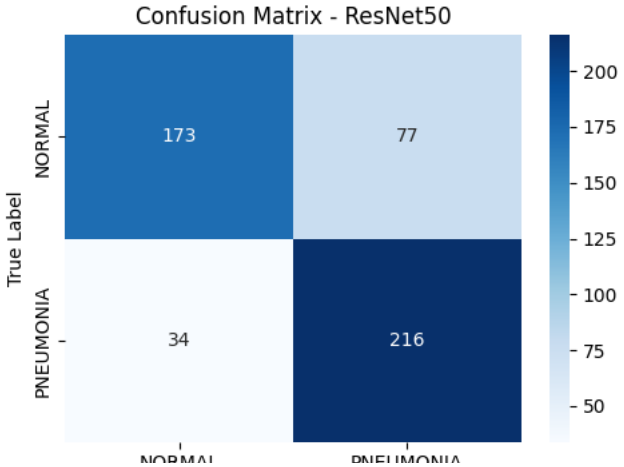
```
 [0.5498047 ]
 [0.02324721]]

Evaluating AlexNet...
16/16 ━━━━━━━━━━━━━━━━━━━━ 8s 471ms/step
[[0.9011226 ]
 [0.31222022]
 [0.6427091 ]
 [0.8227879 ]
 [0.40838122]
 [0.9760286 ]
 [0.89154524]
 [0.84374547]
 [0.88596725]
 [0.35412812]]

Evaluating DenseNet...
16/16 ━━━━━━━━━━━━━━━━━━━━ 32s 1s/step
[[0.04686393]
 [0.2846453 ]
 [0.33226162]
 [0.58352923]
 [0.97902304]
 [0.53983945]
 [0.45525002]
 [0.9751726 ]
 [0.36062115]
 [0.080716  ]]
```

```
# make results into dataframe
results_df = pd.DataFrame([
    {
        'Model': model_name,
        'Accuracy': f"{results['accuracy']:.4f}",
        'Precision': f"{results['precision']:.4f}",
        'Recall': f"{results['recall']:.4f}",
        'F1-Score': f"{results['f1_score']:.4f}"
    }
    for model_name, results in Testing.items()
])
```

```
results_df
```

	Model	Accuracy	Precision	Recall	F1-Score
0	ResNet50	0.7780	0.7372	0.8640	0.7956
1	VGG16	0.8340	0.7554	0.9880	0.8562
2	AlexNet	0.6900	0.6240	0.9560	0.7551
3	DenseNet	0.7960	0.7139	0.9880	0.8289

```
#confusion matrix
for model_name, results in Testing.items():
    cm = results['confusion_matrix']
    plt.figure(figsize=(5, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=['NORMAL', 'PNEUMONIA'],
                yticklabels=['NORMAL', 'PNEUMONIA'])
    plt.title(f"Confusion Matrix - {model_name}")
    plt.xlabel("Predicted Label")
    plt.ylabel("True Label")
    plt.tight_layout()
    plt.show()
```


Confusion Matrix - ResNet50
True Label
NORMAL
PNEUMONIA
173
77
34
216
200
175
150
125
100
75
50